Merdijan Nuhija & Emilija Antic

QuickSplit

Mobile App zur Aufteilung gemeinsamer Ausgaben

Datum: 12.05.2026

Inhalt

1. Einleitung 5
2. Projektidee und Zielsetzung 5
2.1 Problemstellung 5
2.2 Zielsetzung 5
2.3 Zielgruppe 5
3. Anforderungen an die App 6
3.1 Funktionale Anforderungen 6
3.2 Nicht-funktionale Anforderungen 6
4. Lösungskonzept 6
4.1 Wahl der Technologien 6
4.2 React Native 7
4.3 Expo 7
4.4 Firebase 7
4.5 AsyncStorage 8
4.6 Architektur der Anwendung 9
5. Technische Realisierung 9
5.1 Projektstruktur 9
5.2 Beschreibung der wichtigsten Dateien 10
5.3 Navigation 10
5.4 Startscreen und Routing 11
5.5 Benutzerregistrierung und Login 11
5.6 Validierung der Eingaben 11
5.7 Gruppenverwaltung 11
5.8 Gruppen erstellen 12
5.9 Gruppen löschen 12
5.10 Gruppen-Screen 12
5.11 Personen hinzufügen 12
5.12 Speicherung von Ausgaben 13
5.13 Laden und Speichern der Daten 13
5.14 Berechnungslogik 13
5.15 Berechnung der Schulden 13

5.16 Resultat-Screen 14
5.17 Einstellungen 14
5.18 Mehrsprachigkeit 14
5.19 Dark Mode 15
5.20 Währungssystem 15
5.21 Geolocation 15
5.23 State Management 16
5.24 Sicherheit 16
5.25 Performance 16
6. Datenhaltung und Firebase 17
6.1 Firebase Firestore 17
6.2 Datenmodell 17
6.3 Online- und Offline-Szenarien 17
Online 17
Offline 18
Spezialfall 18
7. Benutzeroberfläche und UX 18
7.1 Designentscheidungen 18
7.2 EN-9241-110 19
Aufgabenangemessenheit 19
Selbstbeschreibungsfähigkeit 19
Fehlertoleranz 19
Lernförderlichkeit 19
7.3 Dark Mode 19
7.4 Responsive Darstellung 19
8. Testkonzept und Testergebnisse 19
8.1 Testumgebung 19
8.2 Funktionale Tests 20
8.3 Nicht-funktionale Tests 20
8.4 Fehleranalyse 20
9. Veröffentlichung und Rahmenbedingungen 20
10. Schwierigkeiten und Lösungen 20

11. Fazit 21

1. Einleitung

Im Rahmen des Moduls 335 wurde die mobile Anwendung „QuickSplit“ entwickelt. Die App dient der einfachen Aufteilung gemeinsamer Ausgaben innerhalb von Gruppen.

Bei gemeinsamen Aktivitäten wie Reisen, Restaurantbesuchen oder Ausflügen entstehen häufig gemeinsame Kosten. Die manuelle Aufteilung dieser Beträge führt oft zu Fehlern oder Unübersichtlichkeit. Ziel der Anwendung war deshalb die Entwicklung einer einfachen und modernen Lösung, welche die Kosten automatisch berechnet und übersichtlich darstellt.

Die Anwendung wurde als Cross-Platform-App mit React Native und Expo entwickelt und verwendet Firebase als Backend-Lösung.

2. Projektidee und Zielsetzung

2.1 Problemstellung

Bei gemeinsamen Aktivitäten entstehen häufig gemeinsame Kosten. Die manuelle Aufteilung dieser Ausgaben ist oft kompliziert und fehleranfällig. Besonders bei mehreren Personen verliert man schnell den Überblick darüber, wer wie viel bezahlt hat und welche Beträge noch offen sind.

Viele bestehende Lösungen bieten sehr viele Funktionen und wirken dadurch kompliziert oder unübersichtlich.

2.2 Zielsetzung

QuickSplit soll eine einfache und intuitive Möglichkeit bieten, gemeinsame Ausgaben zu verwalten und automatisch aufzuteilen.

Die wichtigsten Ziele der Anwendung:

- Gruppen erstellen
- Personen hinzufügen
- Ausgaben erfassen
- Kosten automatisch berechnen
- offene Beträge anzeigen
- einfache Bedienung
- moderne Benutzeroberfläche

Der Fokus lag dabei bewusst auf einer minimalistischen Benutzeroberfläche und einer einfachen Bedienung.

2.3 Zielgruppe

Die Zielgruppe der App umfasst:

- Freundesgruppen
- Lernende und Studierende
- Reisende
- Personen bei gemeinsamen Aktivitäten
- kleine Gruppen ohne komplexe Buchhaltung

3. Anforderungen an die App

3.1 Funktionale Anforderungen

Funktion	**Beschreibung**
Registrierung	Benutzerkonto erstellen
Login	Anmeldung über Firebase
Logout	Benutzer abmelden
Gruppen erstellen	Neue Gruppen anlegen
Gruppen löschen	Gruppen entfernen
Personen hinzufügen	Teilnehmer erfassen
Ausgaben erfassen	Beschreibung, Betrag und Zahler speichern
Kostenberechnung	Automatische Berechnung offener Beträge
Resultatanzeige	Übersicht „Wer schuldet wem?“
Mehrsprachigkeit	Deutsch und Englisch
Themes	Hell- und Dunkelmodus
Währungen	CHF, EUR, USD, GBP
Geolocation	Automatische Währungserkennung

3.2 Nicht-funktionale Anforderungen

Anforderung	Beschreibung
Benutzerfreundlichkeit	Intuitive Bedienung
Performance	Schnelle Reaktionszeit
Wartbarkeit	Klare Projektstruktur
Portabilität	Android und iOS
Sicherheit	Firebase Authentication
Ergonomie	Mobile optimierte Oberfläche

4. Lösungskonzept

4.1 Wahl der Technologien

Für die Entwicklung der mobilen Applikation QuickSplit wurde bewusst ein moderner Cross-Platform-Technologie-Stack gewählt. Ziel war eine Lösung, die auf mehreren mobilen Plattformen funktioniert und gleichzeitig eine schnelle Entwicklung sowie einfache Wartbarkeit ermöglicht.

Verwendete Technologien:

- React Native
- Expo
- Firebase
- Firestore
- AsyncStorage
- Expo Router

Die Technologien ermöglichen eine plattformübergreifende Entwicklung und eine einfache Erweiterbarkeit der Anwendung.

4.2 React Native

React Native wurde gewählt, da damit mobile Anwendungen mit JavaScript und React entwickelt werden können. Dadurch kann dieselbe Codebasis sowohl für Android als auch für iOS verwendet werden.

Vorteile von React Native:

- Plattformübergreifende Entwicklung
- Wiederverwendbarer Code
- Moderne Komponentenarchitektur
- Gute Performance
- Grosse Community
- Einfache Erweiterbarkeit

Durch React Native konnte die gesamte Benutzeroberfläche modular aufgebaut werden.

4.3 Expo

Expo wurde verwendet, um die Entwicklung mobiler Anwendungen zu vereinfachen.

Expo bietet viele Vorteile:

- Einfacher Projektstart
- Hot Reload
- Testen direkt auf dem Smartphone
- Einfache Nutzung mobiler APIs
- Schnellere Entwicklung
- Gute Integration mit React Native

Die Anwendung konnte dadurch direkt auf dem Smartphone mit Expo Go getestet werden.

4.4 Firebase

Firebase wurde als Backend-Lösung eingesetzt.

Verwendete Firebase-Dienste:

- Firebase Authentication
- Firebase Firestore

Firebase ermöglicht:

- Benutzerregistrierung
- Login-System
- Cloud-Datenbank
- Speicherung von Gruppen
- Speicherung von Ausgaben
- Synchronisation der Daten

Die Verbindung zu Firebase wird zentral in der Datei firebase.ts aufgebaut:

D:\m335\app\(tabs)_layout.tsx

```
import { getApp, getApps, initializeApp } from "firebase/app";
import {
  browserSessionPersistence,
  getAuth,
  setPersistence,
} from "firebase/auth";
import { getFirestore } from "firebase/firestore";

const firebaseConfig = {
  apiKey: "AIzaSyAL65hteWyQGNuIR3auYRRS7uHOxfZhcgM",
  authDomain: "qicksplit-9476e.firebaseapp.com",
  projectId: "qicksplit-9476e",
  storageBucket: "qicksplit-9476e.firebasestorage.app",
  messagingSenderId: "754701756663",
  appId: "1:754701756663:web:495855bb0c37ab9b970bf2",
};

const app = getApps().length ? getApp() : initializeApp(firebaseConfig);

export const auth = getAuth(app);

// Auf Web bleibt der Login nur im aktuellen Tab/Fenster.
// Dadurch kannst du in mehreren Tabs unterschiedliche Accounts verwenden.
if (typeof window !== "undefined") {
  setPersistence(auth, browserSessionPersistence).catch((error) => {
    console.log("Persistence konnte nicht gesetzt werden:", error);
  });
}

export const db = getFirestore(app);
```

4.5 AsyncStorage

Für lokale Daten wurde AsyncStorage verwendet.

AsyncStorage speichert Einstellungen direkt lokal auf dem Gerät.

Gespeicherte Informationen:

- Sprache
- Theme

- Währung
- Einstellungen des Benutzers

Dadurch bleiben Einstellungen auch nach dem Neustart der App erhalten.

4.6 Architektur der Anwendung

Die App wurde komponentenbasiert aufgebaut.

Jeder Screen besitzt eine eigene Datei und eigene Zuständigkeiten.

Vorteile:

- klare Trennung der Funktionen
- bessere Wartbarkeit
- übersichtlicher Code
- einfache Erweiterbarkeit

5. Technische Realisierung

5.1 Projektstruktur

Die Anwendung wurde übersichtlich in mehrere Ordner und Dateien aufgeteilt.

Projektstruktur:

```
M335/
│
├── app/
│   ├── (tabs)/
│   │   ├── _layout.tsx
│   │   ├── groups.tsx
│   │   ├── index.tsx
│   │   └── settings.tsx
│   │
│   ├── _layout.tsx
│   ├── group.tsx
│   ├── index.tsx
│   ├── login.tsx
│   └── result.tsx
│
├── assets/
├── components/
├── constants/
├── hooks/
├── scripts/
├── services/
│   └── storageServices.ts
│
├── firebase.ts
├── app.json
├── eslint.config.js
└── expo-env.d.ts
```

5.2 Beschreibung der wichtigsten Dateien

Datei	Funktion
login.tsx	Login und Registrierung
groups.tsx	Gruppenübersicht
group.tsx	Gruppenverwaltung
result.tsx	Berechnungen
settings.tsx	Einstellungen
Firebase.ts	Verbindung zu Firebase

5.3 Navigation

Die Navigation wurde mit Expo Router umgesetzt.

Verwendete Navigationsarten:

- Stack Navigation
- Tab Navigation
- Screen Routing

Die Hauptnavigation befindet sich innerhalb des (tabs)-Ordners.

Die Tab-Navigation wird in _layout.tsx definiert.

Die Root-Navigation wird ebenfalls über _layout.tsx gesteuert.

5.4 Startscreen und Routing

Beim Start der Anwendung wird der Benutzer automatisch auf den Login-Screen weitergeleitet.

Die Weiterleitung erfolgt über Expo Router.

Dadurch wird sichergestellt, dass Benutzer zuerst authentifiziert werden.

5.5 Benutzerregistrierung und Login

Die Benutzerverwaltung basiert auf Firebase Authentication.

Folgende Funktionen wurden umgesetzt:

- Registrierung
- Login
- Logout
- Passwortänderung
- Fehlerbehandlung

Der Login-Screen befindet sich in login.tsx.

5.6 Validierung der Eingaben

Bei der Registrierung werden mehrere Eingaben überprüft:

- gültige E-Mail-Adresse
- Passwortlänge
- leere Eingaben
- doppelte Benutzernamen

Zusätzlich besitzt die App eine eigene Fehlerbehandlung für Firebase Authentication.

Beispiele:

Fehler	Reaktion
falsche E-Mail	Fehlermeldung
falsches Passwort	Fehlermeldung
Netzwerkfehler	Fehlermeldung
schwaches Passwort	Fehlermeldung

Dadurch erhält der Benutzer verständliche Rückmeldungen.

5.7 Gruppenverwaltung

Gruppen werden in Firebase Firestore gespeichert.

Jede Gruppe enthält:

- Gruppenname
- Mitglieder
- Personen
- Ausgaben

Die Gruppenübersicht befindet sich in groups.tsx.

5.8 Gruppen erstellen

Neue Gruppen können direkt in der App erstellt werden.

Ablauf:

- Gruppenname eingeben
- Eingabe überprüfen
- Firestore-Dokument erstellen
- Gruppenliste aktualisieren

Die Speicherung erfolgt mit addDoc() aus Firebase Firestore.

5.9 Gruppen löschen

Gruppen können gelöscht werden.

Das Löschen erfolgt über deleteDoc().

5.10 Gruppen-Screen

Im Gruppen-Screen werden:

- Personen verwaltet
- Ausgaben gespeichert
- Zahlungen erfasst
- Anteile berechnet

Die Logik befindet sich in group.tsx.

5.11 Personen hinzufügen

Benutzer können Personen zur Gruppe hinzufügen.

Dabei wird überprüft:

- ob ein Name eingegeben wurde
- ob die Person bereits existiert

Dadurch werden doppelte Einträge verhindert.

5.12 Speicherung von Ausgaben

Jede Ausgabe enthält:

Feld	Beschreibung
title	Beschreibung
amount	Betrag
paidBy	Zahler
shares	Kostenanteile

Die Ausgaben werden in Firebase Firestore gespeichert.

5.13 Laden und Speichern der Daten

Beim Öffnen einer Gruppe werden die Daten aus Firestore geladen.

Beim Ändern werden die Daten erneut gespeichert.

Verwendete Firestore-Funktionen:

- getDoc()
- updateDoc()

Die Daten werden zentral in Firebase gespeichert.

5.14 Berechnungslogik

Die Berechnungslogik bildet die Kernfunktion der Anwendung.

Für jede Person werden berechnet:

- bezahlte Beträge
- konsumierte Beträge
- aktueller Kontostand

Danach werden automatisch:

- Schuldner
- Gläubiger
- offene Beträge

ermittelt.

Die Berechnungslogik befindet sich in result.tsx.

5.15 Berechnung der Schulden

Die Berechnung erfolgt in mehreren Schritten.

Schritt 1

Berechnung aller bezahlten Beträge.

Schritt 2

Berechnung aller konsumierten Beträge.

Schritt 3

Berechnung der Differenz.

Saldo = Bezahlt - Konsumiert

Schritt 4

Ermittlung von:

- creditors
- debtors

Schritt 5

Berechnung der offenen Beträge.

Dadurch wird automatisch berechnet:

„Wer schuldet wem wie viel?“

5.16 Resultat-Screen

Der Resultat-Screen zeigt:

- Gesamtausgaben
- Übersicht pro Person
- offene Beträge
- automatische Abrechnung

Zusätzlich werden positive und negative Beträge unterschiedlich dargestellt.

5.17 Einstellungen

Im Settings-Screen können Benutzer:

- Sprache ändern
- Theme ändern
- Währung ändern
- Passwort ändern
- sich ausloggen

Die Einstellungen befinden sich in settings.tsx.

5.18 Mehrsprachigkeit

Die Anwendung unterstützt:

- Deutsch

- Englisch

Die Übersetzungen werden über Translation-Objekte umgesetzt.

Dadurch können Texte dynamisch angepasst werden.

5.19 Dark Mode

Die App unterstützt Hell- und Dunkelmodus.

Das Theme wird lokal gespeichert und beim Neustart automatisch geladen.

Der Dark Mode verbessert:

- Lesbarkeit
- modernes Design
- Benutzerkomfort

5.20 Währungssystem

Die App unterstützt mehrere Währungen:

- CHF
- EUR
- USD
- GBP

Die Währung kann manuell ausgewählt werden.

Zusätzlich kann die Währung automatisch anhand des Standortes bestimmt werden.

5.21 Geolocation

Für die automatische Währungserkennung wurde Expo Location verwendet.

Ablauf:

- Standortberechtigung anfordern
- Standort abrufen
- Land bestimmen
- passende Währung setzen

Die Standortfunktion wird ausschliesslich verwendet, um die passende Landeswährung automatisch zu bestimmen.

Die Geolocation befindet sich in settings.tsx.

5.22 AsyncStorage

AsyncStorage wurde für lokale Persistenz verwendet.

Gespeichert werden:

- Theme
- Sprache
- Währung
- Einstellungen

Dadurch bleiben Einstellungen auch nach dem Neustart der Anwendung erhalten.

5.23 State Management

Die Zustände der Anwendung werden mit React Hooks verwaltet.

Verwendete Hooks:

- useState()
- useEffect()
- useFocusEffect()

Damit werden:

- Eingaben
- Listen
- Themes
- Spracheinstellungen
- Ausgaben

dynamisch verwaltet.

5.24 Sicherheit

Die Benutzerverwaltung wurde über Firebase Authentication umgesetzt.

Dadurch werden:

- Benutzerkonten verwaltet
- Passwörter sicher verarbeitet
- Logins überprüft

Zusätzlich werden Eingaben validiert, um fehlerhafte Daten zu verhindern.

5.25 Performance

Die Anwendung reagiert auf mobilen Geräten flüssig und ohne auffällige Verzögerungen.

Für Listen wurde FlatList verwendet, um die Darstellung effizient umzusetzen.

Zusätzlich wurde die Benutzeroberfläche bewusst reduziert gestaltet, um eine schnelle Bedienung zu ermöglichen.

6. Datenhaltung und Firebase

6.1 Firebase Firestore

Firestore wurde als Cloud-Datenbank verwendet.

Collections:

Collection	Zweck
groups	Gruppen und Ausgaben
usernames	Benutzerinformation

6.2 Datenmodell

Gruppe

```
{
  "name": "Mallorca",
  "members": [],
  "persons": [],
  "expenses": []
}
```

Ausgabe

```
{
  "title": "Restaurant",
  "amount": 120,
  "paidBy": "max",
  "shares": {}
}
```

6.3 Online- und Offline-Szenarien

Online

Online werden gespeichert:

- Gruppen
- Personen
- Ausgaben
- Benutzerkonten

Die Speicherung erfolgt über Firebase Firestore.

Lokal gespeichert werden:

- Theme
- Sprache
- Währung
- Einstellungen

Dies wurde mit AsyncStorage umgesetzt.

Offline

Im Lokalen speicher bis man wieder Online ist gespeichert (von Firebase geregelt) werden:

- Gruppen
- Personen
- Ausgaben
- Benutzerkonten

Lokal gespeichert werden:

- Theme
- Sprache
- Währung
- Einstellungen

Dies wurde mit AsyncStorage umgesetzt.

Spezialfall

Wenn die App als Guest bedient wird, wird alles im AsyncStorage gespeichert egal ob Online oder Offline.

7. Benutzeroberfläche und UX

7.1 Designentscheidungen

Die Benutzeroberfläche wurde bewusst einfach und übersichtlich gestaltet.

Wichtige Ziele:

- intuitive Bedienung
- schnelle Navigation
- klare Struktur
- mobile Optimierung

Die Oberfläche verwendet:

- grosse Buttons
- klare Farben

- reduzierte Elemente
- gut lesbare Schriftgrössen

7.2 EN-9241-110

Die ergonomischen Prinzipien der EN-9241-110 wurden berücksichtigt.

Aufgabenangemessenheit

Die Anwendung unterstützt direkt die Aufgabe der Kostenaufteilung.

Selbstbeschreibungsfähigkeit

Buttons und Funktionen sind klar beschriftet.

Erwartungskonformität

Die Navigation orientiert sich an bekannten mobilen Anwendungen.

Fehlertoleranz

Falsche Eingaben werden abgefangen und verständlich angezeigt.

Lernförderlichkeit

Die App ist ohne Anleitung verständlich nutzbar.

7.3 Dark Mode

Der Dark Mode verbessert:

Lesbarkeit

modernes Erscheinungsbild

Benutzerkomfort bei Dunkelheit

7.4 Responsive Darstellung

Die Anwendung wurde für mobile Geräte optimiert.

Die Oberfläche passt sich verschiedenen Bildschirmgrössen an.

8. Testkonzept und Testergebnisse

8.1 Testumgebung

Die Anwendung wurde getestet auf:

- Android Smartphone
- Expo Go
- Browser
- Emulator

8.2 Funktionale Tests

Diese sind im Testprotokoll in einem md Names Terstprotokoll.md vorhanden.

8.3 Nicht-funktionale Tests

Bereich	Resultat
Performance	flüssige Bedienung
UX	intuitive Navigation
Stabilität	keine Abstürze
Lesbarkeit	gute Darstellung
Responsiveness	Responsiveness

8.4 Fehleranalyse

Fehler wurden analysiert über:

- Terminal
- Expo Go
- Browser-Konsole
- Visual Studio Code

9. Veröffentlichung und Rahmenbedingungen

Die Anwendung wurde mit Expo entwickelt.

Möglichkeiten zur Bereitstellung:

- Expo Go
- APK Export
- Web-Version

Benötigte Voraussetzungen:

- Internetverbindung für Firebase
- Expo Go oder APK
- Standortberechtigung für automatische Währung

10. Schwierigkeiten und Lösungen

Problem	Lösung
Firebase Authentication Fehler	Eigene Fehlerbehandlung implementiert
Navigation zwischen Screens	Expo Router verwendet
Speicherung lokaler Einstellungen	AsyncStorage verwendet
Mehrsprachigkeit	Translation-System aufgebaut
Berechnung offener Beträge	Eigene Berechnungslogik entwickelt
Datenverwaltung	Firestore Collections verwendet

11. Fazit

Mit QuickSplit konnte eine vollständige mobile Anwendung erfolgreich umgesetzt werden.

Die wichtigsten Ziele wurden erreicht:

- automatische Kostenaufteilung
- moderne Benutzeroberfläche
- Benutzerverwaltung
- Cloud-Datenbank
- Mehrsprachigkeit
- Dark Mode
- mobile Optimierung

Besonders erfolgreich war die Umsetzung mit React Native und Firebase.

Während des Projekts wurden wichtige Kompetenzen im Bereich mobiler Entwicklung erworben:

- React Native
- Expo
- Firebase
- Firestore
- Navigation
- Datenbanken
- State Management
- UX und UI Design

Mögliche zukünftige Erweiterungen:

- Push Notifications
- PDF Export
- Gruppen-Einladungen
- Statistiken
- Echtzeit-Aktualisierung